

shogiAI 資料結構專題報告

棋譜資料庫、程式資料模型、API JSON 與搜尋資料結構

專案：shogiAI

2026 年 5 月

摘要

本報告針對 shogiAI 專案中的資料結構與資料庫架構進行完整說明。專案處理的資料從原始 CSA 棋譜開始，經過 Python 解析、MySQL 正規化儲存、NumPy 訓練資料集、PyTorch checkpoint、HTTP JSON API，最後在前端以棋盤與手順列表呈現。報告從資料結構角度分析：原始檔案格式如何被拆解、資料庫如何設計、每張表的功能與關聯、Python dataclass 如何表示棋子與局面、後端如何將資料序列化 JSON、前端如何保存狀態，以及 AI 搜尋如何使用 transposition table、killer moves、history heuristic 等資料結構。此報告重點不是模型準確率，而是「資料如何被組織、保存、傳輸與操作」。

目录

1	資料結構在本專案中的角色	2
2	原始棋譜資料結構	2
2.1	CSA 棋譜	2
2.2	USI 走法	3
2.3	SFEN 局面	3
3	資料庫資料結構	3
3.1	設計目標	3
3.2	五張主要資料表	3
3.2.1	users	3
3.2.2	players	4
3.2.3	game_records	4
3.2.4	moves	5
3.2.5	positions	5
3.3	資料表關係	5

目錄	2
3.4 為什麼 moves 和 positions 分開	6
4 Python 記憶體資料結構	6
4.1 cshogi.Board	6
4.2 API 中的 dataclass	7
4.2.1 Piece	7
4.2.2 MoveRecord	7
4.2.3 Position	7
4.2.4 Game	8
5 前後端 JSON 資料結構	8
5.1 棋局清單	8
5.2 局面資料	9
5.3 自行對弈請求	9
5.4 AI 對弈請求	10
6 機器學習資料結構	10
6.1 State Tensor	10
6.2 Move Label	10
6.3 Value Label	11
6.4 Metadata	11
7 搜尋演算法資料結構	11
7.1 SearchResult	11
7.2 SearchContext	12
7.3 Transposition Table	12
7.4 Killer Moves	13
7.5 History Heuristic	13
8 前端狀態資料結構	13
8.1 棋譜瀏覽狀態	13
8.2 自行對弈狀態	13
8.3 AI 對弈狀態	14
9 資料結構設計評估	14
9.1 優點	14
9.2 限制	15
10 建議索引設計	15

目录	3
11 結論	15
A 附錄：主要資料流	16

1 資料結構在本專案中的角色

shogiAI 並不是只讀取棋譜並顯示棋盤，而是將不同層級的資料結構串成完整系統：

CSA 文字檔 → Python 物件 → MySQL 關聯表 → NumPy 陣列 → JSON API → DOM 棋盤

因此本專案的資料結構可分成六類：

1. 檔案層資料結構：CSA、USI、SFEN、NPZ、PT。
2. 資料庫層資料結構：users、players、game_records、moves、positions。
3. Python 記憶體資料結構：dataclass、dict、list、cshogi.Board。
4. 機器學習資料結構：state tensor、move label、value label。
5. API 傳輸資料結構：JSON object、array、nested dictionary。
6. 搜尋演算法資料結構：transposition table、SearchContext、SearchResult、killer/history tables。

2 原始棋譜資料結構

2.1 CSA 棋譜

CSA 是文字格式，每一行代表不同類型資料。常見內容包括：

CSA 行	意義
N+	先手棋手名稱。
N-	後手棋手名稱。
\$EVENT	賽事名稱。
\$SITE	比賽地點。
\$START_TIME	對局日期時間。
PI	標準初始局面。
P1 到 P9	棋盤每一列的棋子配置。
+7776FU	先手一手走法。
-3334FU	後手一手走法。
%TORYO	終局結果，例如投了。

CSA 的優點是完整、可讀；缺點是不適合直接查詢。因此專案需要將 CSA 轉換成資料庫與物件結構。

2.2 USI 走法

USI 是程式內部與前後端溝通常用的走法格式，例如：

- 7g7f：從 7g 走到 7f。
- 2b3c+：從 2b 走到 3c 並升變。
- P*5e：打一枚步到 5e。

USI 比 CSA 更適合程式處理，因為它是 cshogi 可直接解析的走法格式。

2.3 SFEN 局面

SFEN 是完整局面的文字表示。專案在資料庫 `positions.sfen` 中保存每一手後的 SFEN。這樣任何局面都可以用：

```
board = cshogi.Board(sfen)
```

直接重建。

3 資料庫資料結構

3.1 設計目標

資料庫設計目標包含：

- 將半結構化 CSA 文字轉成可查詢資料。
- 避免棋手名稱與棋局資訊重複儲存。
- 保存每手走法與每手後局面。
- 讓前端可快速取得任意手數局面。
- 讓數據科學流程可從資料庫匯出訓練樣本。

3.2 五張主要資料表

資料庫由 `src/import_csa_to_db.py` 建立，包含五張表。

3.2.1 users

`users` 表保存匯入者或未來上傳者資訊。

欄位	型別	說明
user_id	INT PK	使用者主鍵。
username	VARCHAR	使用者名稱。
email	VARCHAR	電子郵件，可唯一。
password_hash	VARCHAR	保留登入功能擴充。
role	VARCHAR	使用者角色。
created_at	DATETIME	建立時間。

3.2.2 players

players 保存棋手資料。

欄位	型別	說明
player_id	INT PK	棋手主鍵。
player_name	VARCHAR	棋手姓名。
rank_name	VARCHAR	段位或稱號。
created_at	DATETIME	建立時間。

將棋手獨立成表可避免每盤棋重複儲存棋手名稱，也方便日後統計某棋手對局。

3.2.3 game_records

game_records 是棋局主表。

欄位	型別	說明
game_id	INT PK	棋局主鍵。
uploader_id	INT FK	匯入者。
black_player_id	INT FK	先手棋手。
white_player_id	INT FK	後手棋手。
event_name	VARCHAR	賽事名稱。
site	VARCHAR	比賽地點。
opening	VARCHAR	開局或戰型。
result	VARCHAR	終局結果。
source_format	VARCHAR	來源格式，目前為 CSA。
original_file_name	VARCHAR	原始檔案名稱，用於追蹤與去重。
played_at	DATE	對局日期。
created_at	DATETIME	建立時間。

3.2.4 moves

`moves` 保存每一手棋。

欄位	型別	說明
<code>move_id</code>	INT PK	走法主鍵。
<code>game_id</code>	INT FK	所屬棋局。
<code>move_number</code>	INT	第幾手。
<code>side_to_move</code>	ENUM	先手或後手。
<code>original_move</code>	VARCHAR	原始走法。
<code>usi_move</code>	VARCHAR	USI 格式走法。
<code>move_time</code>	VARCHAR	用時資料。
<code>comment</code>	TEXT	棋譜註解。

3.2.5 positions

`positions` 保存每一手後的完整局面。

欄位	型別	說明
<code>position_id</code>	INT PK	局面主鍵。
<code>game_id</code>	INT FK	所屬棋局。
<code>move_number</code>	INT	第幾手後的局面，0 為初始局面。
<code>side_to_move</code>	ENUM	此局面輪到哪一方下。
<code>sfen</code>	TEXT	完整局面表示。
<code>sfen_hash</code>	CHAR(64)	SFEN 的 SHA-256 hash。
<code>is_check</code>	BOOLEAN	是否被將軍。
<code>legal_moves_count</code>	INT	合法手數量。

3.3 資料表關係

關係如下：

- 一個 `user` 可以上傳多個 `game_records`。
- 一個 `player` 可以出現在多個 `game_records`。
- 一個 `game_record` 對應多個 `moves`。
- 一個 `game_record` 對應多個 `positions`。

概念圖：

$$\text{users} \xrightarrow{1:N} \text{game_records} \xleftarrow{N:1} \text{players}$$

$$\text{game_records} \xrightarrow{1:N} \text{moves} \quad \text{game_records} \xrightarrow{1:N} \text{positions}$$

3.4 為什麼 moves 和 positions 分開

moves 表表示「動作」，positions 表表示「狀態」。這是資料結構上很重要的分離：

$$\text{Position}_t \xrightarrow{\text{Move}_{t+1}} \text{Position}_{t+1}$$

若只保存 moves，查第 N 手局面必須從初始局面 replay 到第 N 手。若保存 positions，前端可以直接查：

```
SELECT sfen
FROM positions
WHERE game_id = ? AND move_number = ?
```

這是用儲存空間換取查詢速度。

4 Python 記憶體資料結構

4.1 cshogi.Board

cshogi.Board 是專案最底層的規則資料結構。它保存：

- 9x9 棋盤。
- 雙方持駒。
- 目前手番。
- 合法手產生器。
- SFEN 轉換。
- Zobrist hash。

常用操作：


```
board = cshogi.Board()
move = board.move_from_usi("7g7f")
board.is_legal(move)
board.push(move)
board.sfen()
board.legal_moves
```

專案不自行實作將棋規則，而是使用 `cshogi` 保證規則正確性。

4.2 API 中的 dataclass

`src/csa_browser_api.py` 定義多個 dataclass。

4.2.1 Piece

```
Piece(color: str, kind: str)
```

表示前端棋盤上的一枚棋子：

- `color`：+ 或 -。
- `kind`：CSA 棋子代碼，例如 FU、HI、OU。

4.2.2 MoveRecord

```
MoveRecord(
    ply,
    color,
    from_square,
    to_square,
    piece,
    usi_like,
    captured
)
```

保存一手棋的前端展示資訊。

4.2.3 Position

```
Position(
    ply,
    turn,
```

```
board,  
hands,  
last_move  
)
```

保存某一手後的完整局面。

4.2.4 Game

```
Game(  
  id,  
  name,  
  metadata,  
  positions,  
  moves  
)
```

本機 CSA 模式會將整盤棋解析成 Game，其中 positions 是局面序列。

5 前後端 JSON 資料結構

5.1 棋局清單

GET /api/games 回傳：

```
{  
  "games": [  
    {  
      "id": "12",  
      "name": "2024 Event A vs B",  
      "moves": 105,  
      "black": "A",  
      "white": "B",  
      "event": "...",  
      "opening": "...",  
      "result": "TORYO",  
      "startTime": "2024-01-01"  
    }  
  ]  
}
```

5.2 局面資料

GET /api/games/<id>?ply=N 回傳：

```
{
  "game": {...},
  "ply": 30,
  "maxPly": 105,
  "turn": "+",
  "turnLabel": "先手",
  "board": [...], ...],
  "hands": {
    "+": {"FU": 1, "KA": 0},
    "-": {"FU": 0, "KA": 1}
  },
  "handOrder": ["HI", "KA", "KI", "GI", "KE", "KY", "FU"],
  "lastMove": {...},
  "moves": [...],
  "isCheck": false,
  "legalMovesCount": 42
}
```

其中 board 是 9x9 陣列，每格包含：

```
{
  "square": "77",
  "piece": {
    "color": "+",
    "kind": "FU",
    "label": "步",
    "promoted": false
  }
}
```

5.3 自行對弈請求

前端送出：

```
{
  "moves": ["7g7f", "3c3d", "2g2f"]
}
```

後端使用 `replay_usi_moves` 重播整個走法序列，再回傳目前局面、合法手與結果。

5.4 AI 對弈請求

```
{
  "moves": ["7g7f", "3c3d"],
  "playerSide": "+",
  "depth": 3,
  "timeLimitMs": 1000
}
```

後端回傳 AI 搜尋資訊：

```
{
  "search": {
    "score": 120,
    "depth": 3,
    "nodes": 5342,
    "timedOut": false,
    "pv": ["2g2f", "8c8d"]
  }
}
```

6 機器學習資料結構

6.1 State Tensor

模型輸入：

$$states \in \mathbb{R}^{N \times 43 \times 9 \times 9}$$

這是四維陣列：

- N ：樣本數。
- 43：特徵平面。
- 9：棋盤列。
- 9：棋盤行。

6.2 Move Label

$$moves \in \mathbb{Z}^N$$

每個值是 0 到 13688 之間的整數，代表 policy 分類標籤。

6.3 Value Label

$$values \in \mathbb{R}^N$$

每個值是：

$$-1, \quad 0, \quad 1$$

6.4 Metadata

meta 保存 JSON 字串，用於追蹤樣本來源：

```
{
  "game_id": 12,
  "ply": 31,
  "sfen": "...",
  "move_usi": "7g7f",
  "result": "TORYO",
  "winner": "black"
}
```

這讓模型資料不是黑盒資料，而是可追蹤回原始棋局與手數。

7 搜尋演算法資料結構

7.1 SearchResult

```
SearchResult(
  move,
  score,
  depth,
  nodes,
  pv,
  timed_out
)
```

保存一次 AI 搜尋的結果：

- 最佳手。
- 分數。

- 完成深度。
- 搜尋節點數。
- principal variation。
- 是否逾時。

7.2 SearchContext

SearchContext 保存搜尋過程共享狀態：

- deadline：時間限制。
- move_orderer：走法排序器，可接 policy model。
- evaluator：局面評估函式。
- nodes：已搜尋節點數。
- tt：transposition table。
- killers：killer move table。
- history：history heuristic table。

7.3 Transposition Table

資料結構：

```
dict[tuple[int, int], TTEntry]
```

key 包含：

- board zobrist hash。
- repetition count。

value 是：

```
TTEntry(depth, score, flag, best_move)
```

用途是避免重複搜尋同一局面。

7.4 Killer Moves

資料結構：

```
dict[int, list[int]]
```

key 是搜尋深度 ply，value 是該深度曾造成 beta cutoff 的安靜手。搜尋同一層其他分支時，這些手會被優先排序。

7.5 History Heuristic

資料結構：

```
dict[int, int]
```

key 是 move，value 是該 move 過去造成剪枝的累積分數。這是一種全域走法排序記憶。

8 前端狀態資料結構

前端 web/app.js 使用多組狀態變數。

8.1 棋譜瀏覽狀態

```
currentGameId  
currentPly  
maxPly  
latestState
```

用途：

- 記錄目前棋局。
- 記錄目前播放到第幾手。
- 保存後端回傳的最新局面 JSON。

8.2 自行對弈狀態

```
selfMoves  
selfRedoMoves  
selfState  
selectedSelfSource
```

用途：

- `selfMoves`：已下走法序列。
- `selfRedoMoves`：復原後可重做的走法。
- `selfState`：目前後端回傳局面。
- `selectedSelfSource`：目前選中的棋盤棋子或持駒。

8.3 AI 對弈狀態

```
aiMoves  
aiState  
aiSelectedSource  
aiThinking  
aiResignedSide
```

用途：

- 保存 AI 對弈走法序列。
- 控制 AI 思考中不可重複點擊。
- 保存玩家是否投降。
- 管理玩家點棋盤或持駒時的選取狀態。

9 資料結構設計評估

9.1 優點

1. **狀態與動作分離**：positions 保存狀態，moves 保存動作，符合棋局轉移模型。
2. **資料可追蹤**：訓練樣本 metadata 可追蹤回 game id、ply、SFEN 與走法。
3. **前後端解耦**：前端只依賴 JSON 格式，不依賴 MySQL 或 CSA 內部細節。
4. **資料來源抽象**：GameSource 使 MySQL 與本機 CSA 檔可替換。
5. **搜尋資料結構完整**：TT、killer、history 都能提升 alpha-beta 搜尋效率。

9.2 限制

1. positions 表會增加儲存空間，因為每手都保存 SFEN。
2. 前端狀態目前以全域變數管理，功能再擴充時可考慮更結構化狀態管理。
3. MySQL 尚未在程式中自動建立索引，資料量變大後需要索引優化。
4. JSON 傳輸每次回傳完整 moves 列表，超長棋局可再做分頁或快取。

10 建議索引設計

若資料量增加，建議加入：

```
CREATE INDEX idx_game_records_played_at
ON game_records(played_at);

CREATE INDEX idx_game_records_original_file_name
ON game_records(original_file_name);

CREATE INDEX idx_moves_game_number
ON moves(game_id, move_number);

CREATE INDEX idx_positions_game_number
ON positions(game_id, move_number);

CREATE INDEX idx_positions_sfen_hash
ON positions(sfen_hash);
```

其中 idx_positions_game_number 對前端查第 N 手局面最重要。

11 結論

shogiAI 的資料結構設計核心是將棋局拆成「棋局主資料、棋手、走法、局面、模型樣本、API JSON、前端狀態、搜尋快取」等不同層級。這種設計讓系統可以同時滿足資料庫查詢、棋譜瀏覽、模型訓練與 AI 搜尋需求。從資料結構角度看，本專案最重要的設計是 moves 與 positions 分離，因為它把棋類問題明確表示成狀態轉移模型：

$$S_t + A_{t+1} \rightarrow S_{t+1}$$

這個模型不只支援前端播放，也支援機器學習中的「局面到下一手」監督式學習。整體而言，shogiAI 是一個以資料結構為基礎，向資料庫、AI 模型與網頁應用延伸的完整系統。

A 附錄：主要資料流

CSA file

- > CSA.Parser
- > cshogi.Board replay
- > MySQL game_records / moves / positions
- > export_policy_dataset.py
- > NumPy states / moves / values
- > PyTorch model
- > csa_browser_api.py JSON
- > web/app.js DOM render